

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192524098
Name	Divyasree.M

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I Divyasree.M/192524098 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Divyasree.M

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and	Most constraints identified	Some constraints identified	Constraints poorly understood

		prioritized accurately			
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

Question 1

Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

1. Constraints Identified & Prioritized

- **Constraint A (Hard SLA):** Availability $> 99.9\%$ (≈ 8.76 hours downtime/year) — highest priority because the platform is user-facing and any NameNode or DataNode outage directly stops uploads/streaming.
- **Constraint B (Hard limit):** Replication factor ≤ 3 — a hard ceiling; availability and durability cannot simply be bought with more replicas.
- **Constraint C (Cost):** Cost-efficient storage growth — media (video/image) volume grows continuously, so raw $3\times$ replication on all data is economically unsustainable long term.

2. System Design

The design combines HDFS High Availability (HA) for the metadata plane with storage-policy based tiering and erasure coding for the data plane, so the two hard constraints (availability, $RF \leq 3$) and the soft constraint (cost) are satisfied simultaneously rather than traded off against each other.

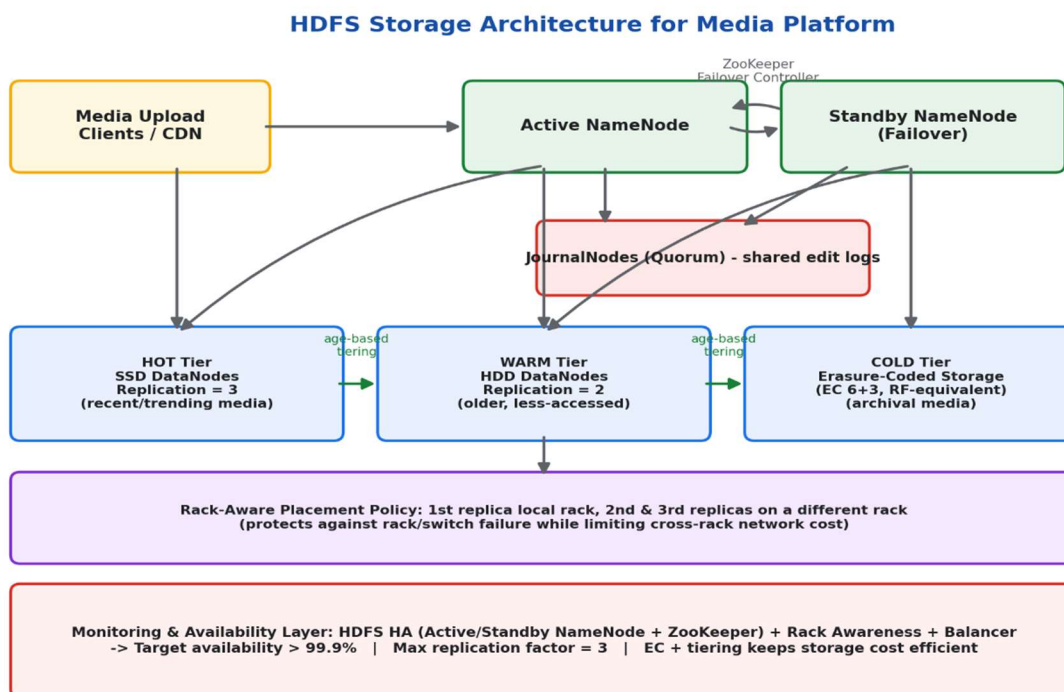


Figure 1: HDFS storage architecture for the media platform

Metadata / Control Plane

- An Active–Standby NameNode pair, coordinated by ZooKeeper Failover Controllers, removes the single point of failure that traditionally caps HDFS availability.
- A JournalNode quorum stores the shared edit log so the Standby NameNode can take over within seconds of an Active failure, with no data loss (fencing prevents split-brain).

Data Plane — Tiered Storage Policies

- **Hot:** HOT tier (recently uploaded / trending media): SSD-backed DataNodes, replication factor = 3 for maximum read throughput and fault tolerance where it matters most.
- **Warm:** WARM tier (older, less-accessed media): HDD-backed DataNodes, replication factor = 2, automatically demoted by HDFS Storage Policies + the Mover tool based on access age.
- **Cold:** COLD tier (archival media): Erasure Coding (e.g., RS-6-3) instead of full replication — it gives fault tolerance equivalent to triple replication at roughly half the storage overhead.
- **Placement policy:** Rack-aware block placement keeps the first replica local and places the remaining replicas on a different rack, protecting against rack/switch failure without exceeding RF = 3.

3. Application of Hadoop Concepts

The design directly applies HDFS HA (NameNode/JournalNode/ZooKeeper), the default rack-aware Block Placement Policy, Heterogeneous Storage + Storage Policies (HOT/WARM/COLD), the Mover/Balancer utilities for lifecycle automation, and Erasure Coding as an HDFS-native alternative to replication.

4. Justification

- HA NameNodes + JournalNode quorum eliminate the historical single point of failure in HDFS, which is what pushes cluster availability above 99.9% — this satisfies Constraint A without violating Constraint B.
- Rack-aware RF = 3 in the hot tier is the maximum allowed by the constraint, and is reserved only for the data that needs it most (frequently accessed media), so the availability benefit is obtained where it has the greatest impact.
- Erasure Coding in the cold tier cuts storage overhead from 200% (RF 3) to about 50% (6+3 EC) while keeping comparable durability, directly satisfying the cost-efficiency constraint as archival volume grows.
- Automated tiering (Storage Policies + Mover) keeps the cost-saving migration continuous and hands-off, rather than a one-time manual fix.

Question 2

A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

1. Constraints Identified & Prioritized

- **Constraint A (Time):** Hard time SLA — the entire job (map, shuffle/sort, reduce, output write) must complete inside a 20-minute batch window.
- **Constraint B (Resources):** Limited cluster nodes — parallelism is bounded, so the design must extract maximum throughput per node rather than assume elastic scale-out.

2. System Design & Task Distribution

The job is designed as a time-budgeted pipeline: input split sizing is matched to the number of available map slots so that the cluster reaches full parallelism in a single wave, and each phase of the job is given an explicit slice of the 20-minute window so bottlenecks are caught early.

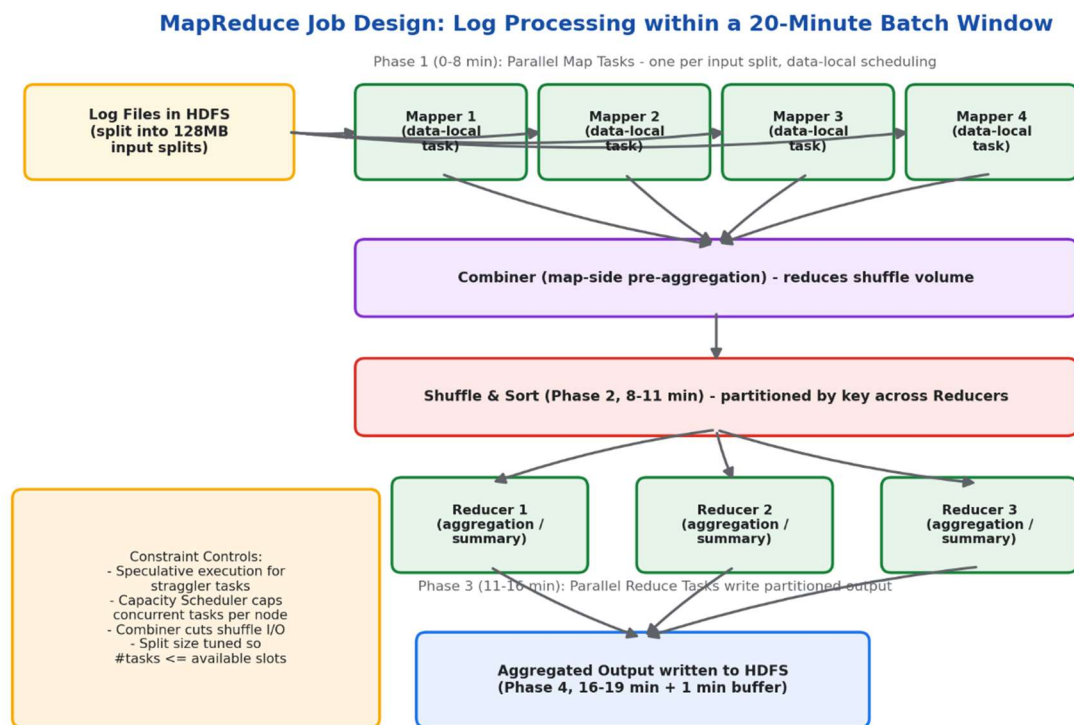


Figure 2: MapReduce job design and phase-wise task distribution within the 20-minute window

Phase-wise Time Budget

- Phase 1 — Map (0–8 min): log file is split into ~128 MB input splits, sized so the number of map tasks equals (or slightly exceeds) the number of available map slots — this avoids a second wave of scheduling delay.
- Phase 2 — Shuffle & Sort (8–11 min): a Combiner performs map-side pre-aggregation so far less intermediate data crosses the network; intermediate output is compressed (e.g., Snappy) to shrink shuffle I/O further.

- Phase 3 — Reduce (11–16 min): the number of reducers is capped to match available reduce slots so tasks run in one wave; partitioning is key-based for balanced load across reducers.
- Phase 4 — Output write + buffer (16–20 min): aggregated results are written back to HDFS with 4 minutes reserved as a safety buffer for stragglers.

3. Application of Hadoop Concepts

The design uses input-split sizing and data-local task scheduling, a Combiner for map-side aggregation, intermediate-data compression, the Capacity Scheduler to guarantee slot availability for this batch job, and speculative execution to relaunch slow (straggler) tasks on limited hardware without adding nodes.

4. Justification

- Matching split count to available slots guarantees single-wave scheduling — with limited nodes, a second wave of tasks alone could consume several minutes and blow the SLA.
- The Combiner and compressed intermediate data reduce shuffle volume, which is usually the biggest variable-time phase; this keeps Phase 2 predictable even under limited network/disk bandwidth.
- Speculative execution is the correct tool for a node-limited cluster because it recovers from stragglers (slow disks, transient contention) without requiring extra hardware — critical since the constraint rules out simply adding nodes.
- Reserving an explicit output/buffer window (rather than assuming reduce finishes exactly on time) builds resilience into the SLA itself.

Question 3

A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

1. Constraints Identified & Prioritized

- **Constraint A (Priority):** ETL must receive priority (guaranteed resources and, when necessary, the ability to preempt lower-priority work) so its SLA is protected.
- **Constraint B (Fairness):** Fairness must still be preserved — Reporting and ML jobs cannot be starved indefinitely, and jobs of equal priority within a workload must share resources fairly.
- **Constraint C (Resilience):** The cluster must recover automatically from node failure without manual operator intervention.

2. System Design

The Capacity Scheduler is used as the base scheduler (because it gives hard, predictable capacity guarantees per workload, which a pure Fair Scheduler cannot) but each queue is configured with a Fair ordering policy internally, so fairness is achieved at both the workload level and the job level.

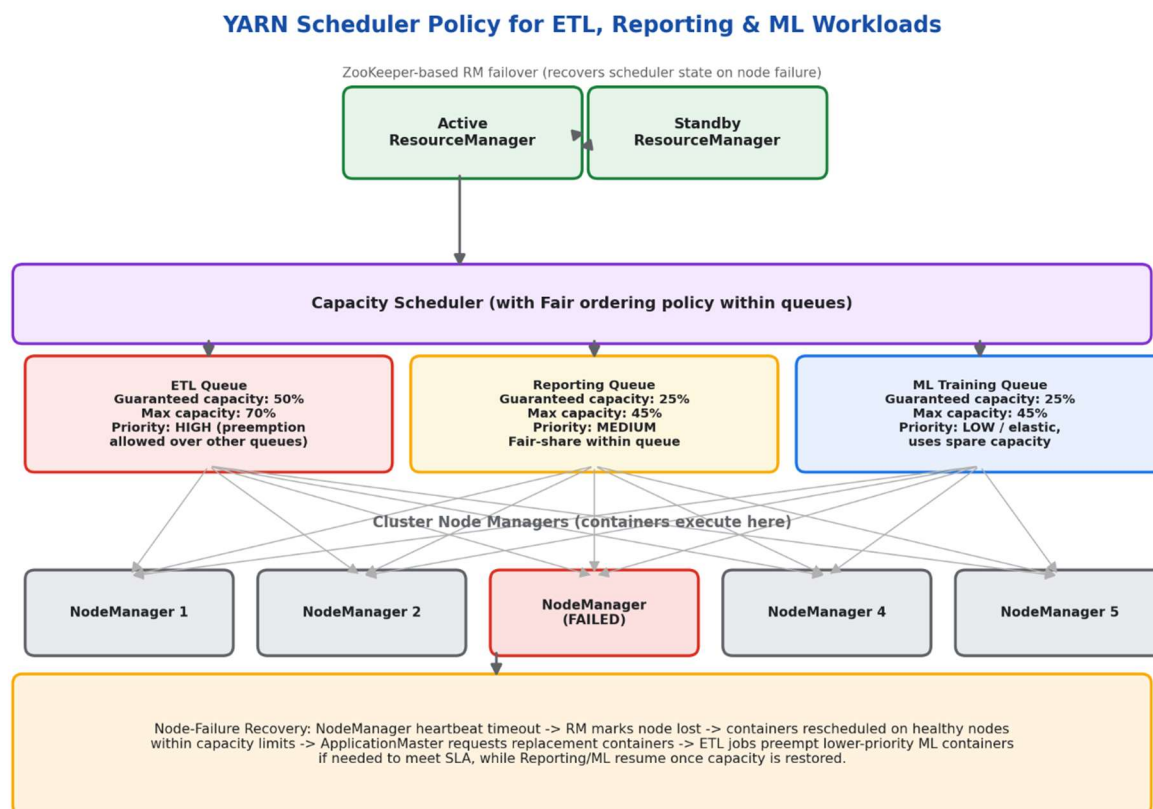


Figure 3: YARN scheduler queue design with priority, fairness, and node-failure recovery

Queue Configuration

- ETL Queue — guaranteed capacity 50%, max capacity 70%, priority HIGH, preemption enabled over other queues so ETL SLA is met even under contention.
- Reporting Queue — guaranteed capacity 25%, max capacity 45%, priority MEDIUM, jobs ordered by the Fair policy so simultaneous reports share resources evenly.
- ML Training Queue — guaranteed capacity 25%, max capacity 45%, priority LOW / elastic, allowed to opportunistically use spare cluster capacity when ETL and Reporting are idle.

Node-Failure Recovery

- The ResourceManager runs as an Active–Standby HA pair coordinated through ZooKeeper, so scheduler state itself survives an RM crash.
- On a NodeManager heartbeat timeout, the ResourceManager marks the node lost, and all containers on that node are rescheduled onto healthy nodes within each queue's capacity limits.
- The affected ApplicationMaster requests replacement containers; if capacity is tight, ETL containers are permitted to preempt lower-priority Reporting/ML containers to protect the ETL SLA, and preempted jobs resume once capacity is restored.

3. Application of Hadoop Concepts

The design applies YARN's Capacity Scheduler with hierarchical queues and preemption, per-queue ordering policies (Fair vs FIFO), ResourceManager High Availability via ZooKeeper, and the standard NodeManager heartbeat/container-reallocation failure-recovery mechanism.

4. Justification

- Capacity guarantees (not just priority weights) are what actually protect an SLA — a 50% guaranteed floor for ETL cannot be eroded by a burst of ML jobs, satisfying Constraint A.
- Max-capacity caps on every queue (70%/45%/45%) prevent any single workload, including ETL, from permanently starving the others, which directly satisfies the fairness constraint.
- RM HA plus automatic container rescheduling on NodeManager failure meets the resilience constraint without operator involvement, and doing this via the scheduler (rather than external tooling) keeps recovery time low and preemption-aware.

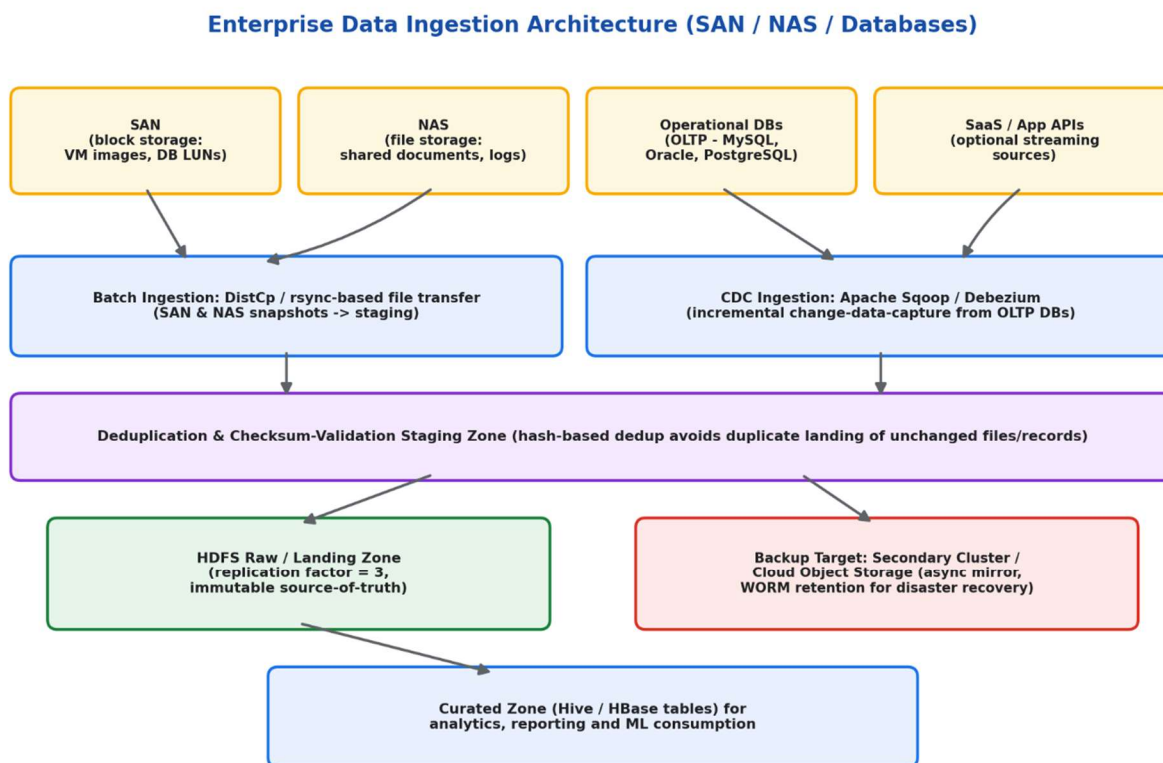
Question 4

An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

1. Constraints Identified & Prioritized

- **Constraint A:** Minimal duplication — the same file or record must not be repeatedly re-copied into the platform once it is already present and unchanged.
- **Constraint B:** Backup reliability — the platform must guarantee a durable, restorable copy of ingested data independent of the primary ingestion pipeline.
- **Constraint C:** Heterogeneous sources — SAN (block storage), NAS (file storage), and operational databases (OLTP) each need a different, appropriate ingestion mechanism.

2. Constrained Architecture



Design meets constraints: minimal duplication (dedup + CDC instead of full re-copy), backup reliability (async DR mirror), auditable lineage

Figure 4: Enterprise ingestion architecture for SAN, NAS, and operational databases

Component-by-Component Design

- SAN / NAS → Batch Ingestion (DistCp / rsync-based transfer): snapshots of block and file storage are pulled into a staging area using parallel, checksum-verified copy tools.

- Operational DBs → CDC Ingestion (Apache Sqoop for scheduled pulls, Debezium for log-based Change-Data-Capture): only inserted/updated/deleted rows since the last run are captured, instead of re-exporting entire tables.
- Deduplication & Checksum-Validation Staging Zone: incoming files/records are hashed and compared against existing landing-zone content; unchanged data is skipped rather than re-landed.
- HDFS Raw/Landing Zone: the deduplicated data is written once with replication factor 3, forming an immutable source-of-truth.
- Backup Target (secondary cluster or cloud object storage): an asynchronous mirror of the landing zone, with WORM (write-once-read-many) retention, dedicated purely to disaster recovery.
- Curated Zone (Hive/HBase tables): cleaned, query-ready data for analytics, reporting, and ML consumption.

3. Application of Hadoop Concepts

The architecture uses DistCp for scalable parallel bulk copy, Sqoop/Debezium for database ingestion via CDC, HDFS's default 3x replication and rack awareness in the landing zone, and a Hive/HBase curated layer as the standard Hadoop-ecosystem consumption pattern.

4. Justification

- CDC ingestion (rather than full re-extraction) is what actually satisfies the minimal-duplication constraint for databases: only deltas move, cutting both duplicate storage and load on production OLTP systems.
- The hash-based dedup/checksum stage catches the SAN/NAS case — repeated snapshots of largely unchanged file shares would otherwise re-land identical objects.
- Making the backup an asynchronous mirror (not a synchronous write) decouples backup latency from ingestion SLA, so backup reliability is achieved without slowing the primary pipeline — and WORM retention protects the backup from accidental or malicious overwrite.
- Keeping the landing zone immutable and separate from the curated zone preserves an auditable, replayable source-of-truth if downstream transformations ever need to be re-run.

Question 5

Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

1. Constraints Identified & Prioritized

- **Constraint A:** Secure data movement — data must be protected both in transit and at rest, with authenticated, authorized access throughout the pipeline.
- **Constraint B:** High availability — no single component (NameNode, ResourceManager, ingestion layer) may be a single point of failure.
- **Constraint C:** Integration with Apache NiFi / ETL pipelines — ingestion must be flow-based, auditable, and cleanly hand off into standard Hadoop processing engines.

2. End-to-End System Design

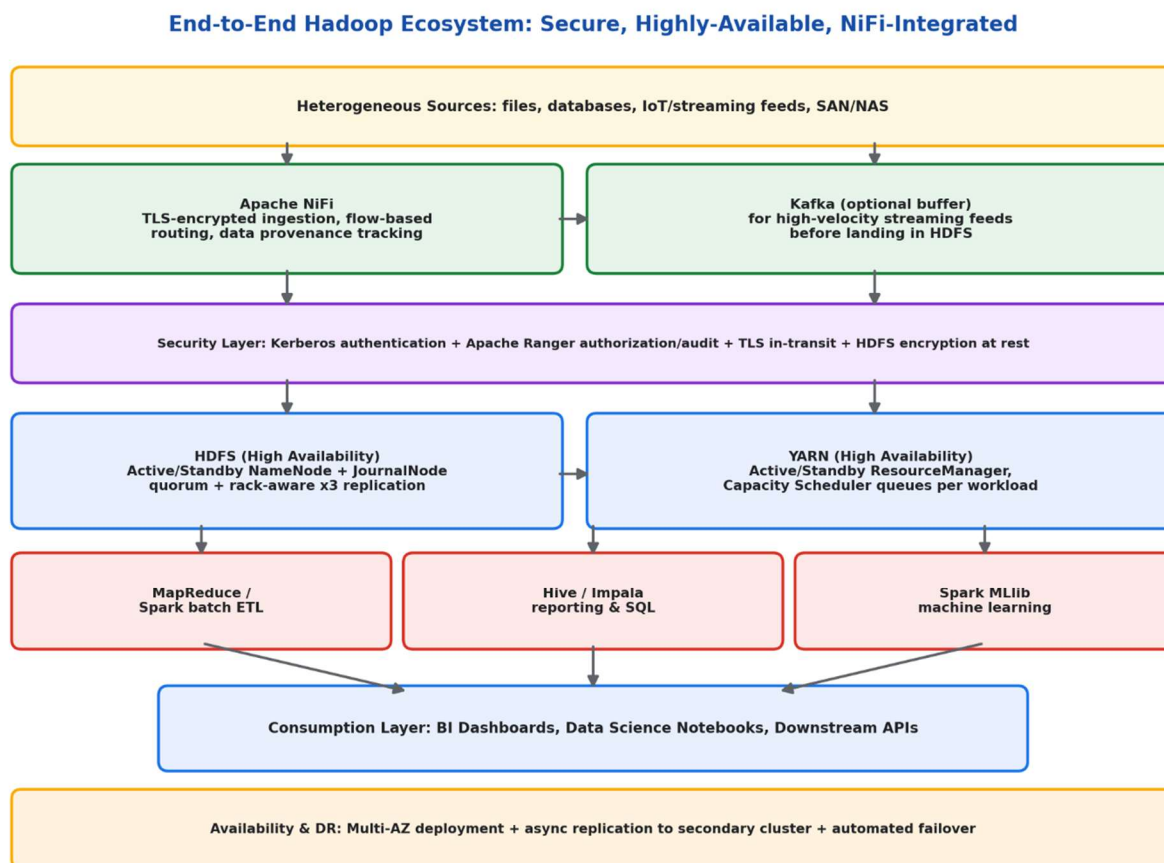


Figure 5: End-to-end secure, highly-available Hadoop ecosystem integrated with Apache NiFi

Layered Design

- Ingestion: Apache NiFi ingests from files, databases, IoT/streaming feeds, and SAN/NAS over TLS, with every flow's lineage tracked via NiFi's built-in data provenance; a Kafka buffer optionally absorbs high-velocity streams before they land in HDFS.
- Security layer: Kerberos handles cluster-wide authentication, Apache Ranger provides fine-grained authorization and audit logging across HDFS/Hive/YARN, TLS protects all in-transit traffic, and HDFS Transparent Data Encryption protects data at rest.
- Storage & resource layer: HDFS runs with Active/Standby NameNodes and a JournalNode quorum (RF = 3, rack-aware); YARN runs with Active/Standby ResourceManagers and Capacity Scheduler queues per workload.
- Processing layer: MapReduce/Spark for batch ETL, Hive/Impala for SQL-based reporting, and Spark MLlib for machine learning — all reading from the same secured, highly-available HDFS.
- Consumption layer: BI dashboards, data-science notebooks, and downstream APIs consume curated output.
- Availability & DR: multi-AZ cluster deployment with asynchronous replication to a secondary cluster and automated failover.

3. Application of Hadoop Concepts

The design integrates Apache NiFi as the ingestion/orchestration layer feeding HDFS, HDFS High Availability and rack-aware replication, YARN High Availability with the Capacity Scheduler, Kerberos/Ranger security integration, and the standard MapReduce/Spark/Hive/MLlib processing stack — covering the full Hadoop ecosystem end to end.

4. How the Design Satisfies Every Constraint

- Secure data movement: TLS encrypts every hop from source to NiFi to HDFS; Kerberos ensures only authenticated principals can move data; Ranger enforces who is authorized to read/write each dataset and logs every access; HDFS TDE keeps data encrypted once it is at rest — together this secures the entire data movement path, not just the network link.
- High availability: HA NameNode/ResourceManager pairs plus JournalNode/ZooKeeper quorum remove metadata- and scheduling-layer single points of failure; multi-AZ deployment with async DR replication protects against a full-site outage, satisfying availability at both the component and the site level.
- NiFi/ETL integration: NiFi's flow-based routing and data provenance give an auditable, low-code ingestion layer that lands data directly into the secured, highly-available HDFS zone, from which Spark/Hive/MLlib pipelines consume it immediately — so ingestion and processing are integrated rather than bolted together.